

MANIFOLD

Un Motor Matemático para Mapear Vulnerabilidades

Proyecto MANIFOLD

Whitepaper v0.1 — borrador de trabajo, no revisado por pares

Abstract

MANIFOLD trata un artefacto de software no como un conjunto de reglas a emparejar, sino como un *objeto matemático*: un grafo tipado y ponderado enriquecido con estructura *algebraica*, *espectral*, *topológica* y *geométrica*. Sobre ese objeto se computa un campo escalar de **potencial de vulnerabilidad** $V(x)$, que hace emerger las regiones anómalas como *rasgos* geométricos / topológicos / espectrales en lugar de coincidencias literales de cadenas. Un **agente LLM autónomo** navega el “manifold” resultante, formula hipótesis sobre esas regiones y despacha **verificadores formales** (SMT, ejecución simbólica, interpretación abstracta) que confirman o refutan cada hipótesis — cerrando el ciclo entre intuición estadística y prueba matemática. Este documento define la representación intermedia, formaliza cada capa matemática, enuncia las leyes de mapeo que conectan rasgos matemáticos con clases de vulnerabilidad, y especifica la arquitectura del agente y la hoja de ruta.

1 Motivación

Los escáneres tradicionales (SAST/DAST) enumeran patrones sintácticos. Son frágiles: pasan por alto vulnerabilidades novedosas y ahogan al analista en falsos positivos. Faltan dos capacidades:

- (1) Una *representación unificada y cuantitativa* de la “estructura del programa” sobre la cual proyectar y fusionar muchas señales *independientes*.
- (2) Un *ciclo de razonamiento* que convierta “región sospechosa” en “vulnerabilidad probada” — el LLM aporta la intuición, la matemática aporta la prueba.

MANIFOLD proporciona ambas.

2 La idea central

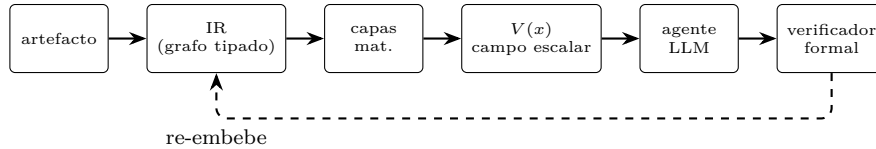


Figure 1: El pipeline: un artefacto se convierte en un grafo tipado, enriquecido con estructura matemática en un campo escalar de vulnerabilidad $V(x)$, que un agente LLM navega; un verificador formal confirma o refuta cada hipótesis y el resultado se re-embebe en el manifold.

Una vulnerabilidad *no* es un patrón; es una *violación de una invariante estructural* (una condición de naturalidad rota, un ciclo persistente, un cuello de botella de curvatura negativa, un flujo de taint que cruza un corte espectral de confianza). Cada capa detecta un *tipo* distinto de violación de invariante, y el campo fusionado $V(x)$ ordena las regiones para que el agente las inspeccione.

3 La Representación Intermedia (IR)

Definición 1 (Grafo de programa). Un *grafo de programa* es una tupla $G = (V, E, \tau_V, \tau_E, \omega)$ donde

- V es un conjunto finito de *nodos* (módulos, funciones, bloques, sentencias, llamadas, variables, fuentes, sumideros);
- $E \subseteq V \times V$ es un conjunto de *aristas* dirigidas;
- $\tau_V : V \rightarrow \mathbb{N}_K$ asigna a cada nodo un *tipo* de $K = \{\text{módulo, función, bloque, sentencia, llamada, asignación, variable, parámetro, fuente, sumidero, gate}\}$;
- $\tau_E : E \rightarrow \mathcal{T}$ asigna a cada arista un *tipo* $\mathcal{T} = \{\text{control, dato, llamada, taint, confianza, auth}\}$;
- $\omega : E \rightarrow \mathbb{R}_{\geq 0}$ asigna a cada arista un *peso* no negativo (por defecto 1).

La IR es la única fuente de verdad. Cada adaptador (código fuente, binario, API web, agente LLM) debe producir un grafo de programa; cada análisis lo consume. El esquema es agnóstico al lenguaje y serializable a JSON (las capas Python y Rust comparten el mismo esquema).

4 Capas matemáticas

Para cada capa fijamos un subgrafo por tipo de arista τ_E , y luego computamos estructura.

4.1 Capa espectral (teoría algebraica de grafos)

Para un tipo de arista t , definimos la *adyacencia* simétrica A_t y el *grado* D_t , y el *Laplaciano combinatorio*

$$L_t = D_t - A_t.$$

L_t es semidefinido positivo; su espectro $0 = \lambda_1 \leq \lambda_2 \leq \dots$ codifica la estructura global:

- λ_2 es la *conectividad algebraica* (valor de Fiedler). Su autovector f_2 — el *vector de Fiedler* — induce un corte canónico $S^+ = \{v : f_2(v) \geq 0\}$.
- El *embedding espectral* $\Phi^{(d)}(v) = (f_2(v), f_3(v), \dots, f_{d+1}(v))$ mapea nodos a $\mathbb{R}^d$.
- La desigualdad de Cheeger acota el cociente de cuello de botella $h(G)$ por $\frac{\lambda_2}{2} \leq h(G) \leq \sqrt{2 d_{\max} \lambda_2}$.

Señal 1 (anomalía espectral). El error de reconstrucción de un nodo bajo una reconstrucción de eigenmap de bajo rango, y su distancia al corte de Fiedler, se usan como puntajes de anomalía.

4.2 Capa topológica (homología persistente)

Del grafo de programa construimos una *filtración* de complejos simpliciales (p. ej. un complejo de Vietoris–Rips o de bandera sobre la métrica de caminos más cortos, o una filtración guiada por ω). La homología persistente produce, para cada dimensión k , un *diagrama de persistencia* Dgm_k de pares nacimiento–muerte. β_0 cuenta componentes, β_1 cuenta ciclos independientes; los puntos lejos de la diagonal tienen alta persistencia. El algoritmo **Mapper** (Singh–Mémoli–Carlsson) construye un *nervio* navegable a partir de una función filtro $f : V \rightarrow \mathbb{R}$, produciendo el mapa que da nombre al proyecto.

4.3 Capa geométrica (curvatura discreta)

La *curvatura de Ollivier–Ricci* de una arista (u, v) es

$$\kappa(u, v) = 1 - \frac{W_1(m_u, m_v)}{d(u, v)},$$

con medidas de paseo aleatorio perezoso m_u, m_v y distancia de Wasserstein-1 W_1 . La curvatura negativa marca *embudos* por los que pasan muchos caminos más cortos. La *curvatura de Forman–Ricci* (sin pesos) es $\kappa_F(u, v) = 4 - \deg(u) - \deg(v)$, marcando de forma barata los nodos de alto grado.

4.4 Capa algebraica (retículos y teoría de categorías)

Interpretación abstracta. El *retículo de taint* $\mathbb{T} = \{\perp = \text{limpio} \sqsubseteq \top = \text{manchado}\}$ (un retículo producto sobre etiquetas) da una *conexión de Galois* (α, γ) entre la semántica recolectora concreta y el dominio abstracto de taint, garantizando solidez. *Teoría de categorías.* El programa es una categoría **Prog**; los efectos son mónadas, la autorización es un funtor $F : \mathbf{Prog} \rightarrow \mathbf{Auth}$. **Ley de diseño (naturalidad del taint):** una vulnerabilidad es un flujo de taint que rompe el cuadrado de naturalidad de F .

4.5 Capa formal (ejecución simbólica y model checking)

La ejecución simbólica calcula una condición de camino Φ y un estado simbólico σ ; un solucionador SMT comprueba $\text{SAT}(\Phi \wedge \text{malo})$, cuyo modelo es un exploit concreto. La alcanzabilidad es la fórmula CTL **EF** malo; la propagación de taint es un autómata finito cuyo lenguaje aceptado es exactamente el conjunto de caminos fuente $\rightarrow$ sumidero.

5 El campo escalar de vulnerabilidad

$$V(x) = \sigma\left(\sum_i \alpha_i s_i(x)\right),$$

donde s_i son las señales normalizadas de cada capa (anomalía espectral, outlier de persistencia, curvatura, altura de retículo, alcanzabilidad formal), α_i son pesos (aprendibles) y σ una compresión logística. V es un ranking, un filtro de Mapper y un campo de costos para la navegación.

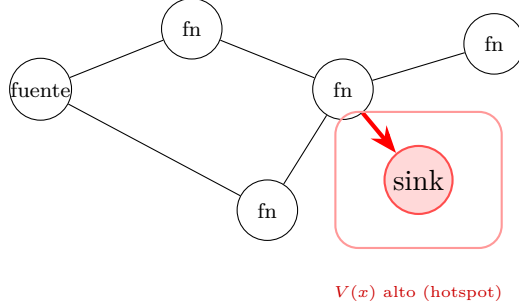


Figure 2: El manifold: los nodos son componentes del programa, las aristas son relaciones tipadas. Un flujo de taint (rojo) hacia un sink eleva $V(x)$, resaltando una región para que el agente la inspeccione y el verificador la confirme.

6 Leyes de mapeo

#	Ley	Clase de vulnerabilidad
L1	Cuello de botella ($\kappa \ll 0$)	escalada de privilegios, bypass de auth
L2	Ciclo (H_1 persistente)	reentrancia, recursión infinita
L3	Corte de confianza (corte de Fiedler)	inyección que cruza la frontera de confianza
L4	Ruptura de naturalidad	violación arbitraria de taint
L5	Outlier de persistencia	rasgo real vs. espurio
L6	Alcanzabilidad ($\text{SAT}(\Phi \wedge \text{malo})$)	exploit concreto

7 El agente LLM autónomo

Un ciclo cerrado: (1) *ingesta* hacia la IR; (2) *embebir y mapear* ($V(x)$, embedding espectral, persistencia, curvatura, Mapper); (3) *hipotetizar* (el LLM propone candidatos desde el manifold); (4) *verificar* (la capa formal prueba o refuta); (5) *refinar* (los hallazgos re-ponderan y re-embeben); (6) *reportar*. La capa de proveedor LLM apunta a cualquier endpoint compatible con OpenAI mediante **LiteLLM**. El LLM no hace detección cruda: la matemática propone regiones, el LLM propone hipótesis, la capa formal propone pruebas.

8 Desafíos de diseño y mitigaciones

Una revisión de diseño identificó cinco riesgos estructurales. Para cada uno enunciamos la mitigación adoptada en el prototipo y el problema abierto.

C1 – La paradoja del cuello de botella. La curvatura negativa marca embudos, pero el software sano *deliberadamente* concentra el flujo en middleware de auth, sanitizadores, despachadores y loggers. Reportarlos como leads de escalada de privilegios produce falsos positivos sistemáticos. *Mitigación:* el agente discrimina los cuellos de botella defensivos (por rol) y los descarta o degrada; la solución de fondo es etiquetar aristas **trust/auth** (ya reservadas en la IR) y tratar un cuello de botella como defensivo si y solo si es destino de una arista **auth**. *Abierto:* aprender la etiqueta defensivo/ofensivo a partir de datos.

C2 – La simetrización destruye la causalidad. El Laplaciano combinatorio $L = D - A$ y la homología persistente estándar operan sobre espacios métricos no dirigidos, pero el flujo de control

y de datos es dirigido: que A manche a B no es lo mismo que B manche a A . Simetrizar puede destruir la dirección del taint. *Mitigación*: el prototipo implementa el Laplaciano dirigido de Chung (componente fuertemente conexa más grande, vector de Perron, conectividad algebraica dirigida), que queda indefinido para un DAG — en sí misma una señal significativa. *Abierto*: complejos de bandera dirigidos / homología de caminos (Grigor’yan–Lin–Muranov–Yau).

C3 – Costo del transporte óptimo. Ollivier–Ricci exige un PL de transporte por arista: $\mathcal{O}(|E| \cdot \text{PL})$, prohibitivo más allá de $\sim 10^5$ aristas. *Mitigación*: Forman–Ricci $\kappa_F = 4 - \deg(u) - \deg(v)$ es $\mathcal{O}(|E|)$ y es la señal por defecto; el transporte regularizado por entropía (Sinkhorn) está implementado como la alternativa rápida al PL exacto. *Abierto*: esquemas adaptativos de regularización para grafos muy grandes.

C4 – Operacionalizar L4 (naturalidad). Se requiere una regla constructiva. Sean las aristas `auth` E_A las que marcan fronteras de privilegio. Definamos el funtor de autorización F como la etiqueta de si un nodo es alcanzable *detrás* de una arista `auth`. Entonces un flujo de taint $s \rightarrow^* k$ *viola la naturalidad* si y solo si su camino cruza al menos una arista `auth` — es decir, datos no confiables entran a una región protegida sin que la frontera los neutralice. Esto es una consulta de alcanzabilidad sobre el grafo combinado ($\text{dato} \cup \text{taint} \cup \text{auth}$), directamente computable y automatizable. *Implementado*: un extractor de autorización emite nodos `gate` y aristas `auth` desde decoradores de frontera de privilegio, y un flujo de taint dentro de una función protegida se reporta como violación de naturalidad.

C5 – Puente manifold-verificador. Los rasgos geométricos son cuantitativos; Z3 necesita precondiciones exactas. Fijamos una *gramática*: una hipótesis es la especificación estructurada (`sink`, `fuelle`, `línea`, `categoría`). El manifold aporta la especificación (conoce `sink` y `línea`), el LLM aporta la interpretación (CWE, descripción, confianza) y el verificador consume la especificación — nunca la prosa del LLM. Esto mantiene la consulta SMT precisa y reproducible.

La tesis de fondo se sostiene: los analizadores estáticos fallan porque buscan palabras en vez de invariantes en una estructura topológica. El criterio es si $V(x)$ poda el espacio de búsqueda para Z3/angr de forma más eficiente que un DFS plano sobre el grafo de llamadas.

9 Estado de implementación

Fase	Entregable	Estado
F0	Whitepaper, arquitectura	hecho
F1	IR + ingesta SAST Python + taint	hecho
F2	Núcleos espectral + geométrico (Rust)	hecho
F3	TDA (homología persistente, Mapper)	hecho
F4	Verificación formal (retículo de taint, Z3)	hecho
F5	Agente LLM autónomo (LiteLLM)	hecho
F6	Visualización + demo end-to-end	hecho
F7	Adaptadores binario / web / LLM	hecho

10 Validación y uso responsable

La validación usa micro-benchmarks de juguete con verdad de referencia conocida, luego corpus etiquetados (OWASP Benchmark, SARD) una vez que F4 aterrice, midiendo precisión/recall del campo fusionado frente al taint crudo. MANIFOLD es una herramienta de defensa / pruebas autorizadas; la liberamos bajo el supuesto de uso autorizado y alentamos la divulgación coordinada.

References

- [1] M. Fiedler, *Algebraic connectivity of graphs*, 1973.
- [2] F. Chung, *Spectral Graph Theory*, 1997.
- [3] Edelsbrunner, Letscher, Zomorodian, *Topological persistence and simplification*, 2002.
- [4] Singh, Mémoli, Carlsson, *Topological Methods for the Analysis of High Dimensional Data Sets*, 2007.
- [5] Y. Ollivier, *Ricci curvature of Markov chains on metric spaces*, 2009.
- [6] P. Cousot & R. Cousot, *Abstract interpretation: a unified lattice model*, 1977.
- [7] S. Mac Lane, *Categories for the Working Mathematician*, 1971.
- [8] J. King, *Symbolic execution and program testing*, 1976.
- [9] E. M. Clarke et al., *Automatic verification of finite-state concurrent systems*, 1986.
- [10] F. Chung, *Laplacians and the Cheeger inequality for directed graphs*, 2005.
- [11] Grigor'yan, Lin, Muranov, Yau, *Homotopy theory for digraphs*, 2012.
- [12] M. Cuturi, *Sinkhorn distances: lightspeed computation of optimal transport*, 2013.